

# Programming in Standard ML

*Robert Harper | Carnegie Mellon University (Course 15-212: Principles of Programming)*

## 1. Foundations, Computation Models, Types, Values & Static Typing Discipline

Standard ML (SML) is a formally defined, statically typed functional programming language. Unlike imperative languages such as C, C++, or Java that rely on an execution model based on sequential command execution and stateful memory mutation, ML computation is rooted in the mathematical evaluation model of expressions. Computation in ML is a direct generalization of algebraic simplification: expressions are evaluated by substituting values for variables and applying deterministic reduction rules according to formal operational semantics.

### 1.1 Evaluation vs. Command Execution Paradigm

- **The Algebraic Evaluation Model:** Expressions in ML denote values. Just as the algebraic polynomial expression  $x^2 + 2x + 1$  evaluates to 9 when 2 is substituted for  $x$ , ML programs compute by simplifying expressions down to canonical irreducible forms called values. This model eliminates implicit temporal dependencies, enabling equational reasoning where code fragments can be replaced by their values without altering program behavior.
- **Subsumption of Imperative Programming:** The evaluation model naturally subsumes the imperative state-modification model as a specialized sub-case. Rather than forcing all computation into memory-mutation commands, ML treats expression evaluation as primary. Stateful memory allocation, reading, and assignment are introduced as explicit, encapsulated computational effects without destroying the mathematical elegance of effect-free code.
- **Benefits for Verification & Reasoning:** Because computation is mathematical simplification, programs are amenable to rigorous proofs of correctness via mathematical and structural induction. This eliminates entire classes of runtime errors (such as null pointer dereferences, uninitialized variable reads, and unintended shared-memory aliasing bugs) before runtime execution.

### 1.2 The Triad of Expression Characteristics: Types, Values, and Effects

Every syntactic expression in Standard ML is formally characterized across three fundamental dimensions:

- **1. Type (Classification & Prediction):** An expression may or may not possess a static type. A type serves as a static prediction of the form of value the expression will yield if evaluation terminates. An expression is well-typed if it satisfies the formal typing rules of the language; ill-typed expressions are rejected at compile time and are ineligible for evaluation.
- **2. Value (Evaluation Outcome):** A well-typed expression may or may not yield a value. Evaluation may fail to produce a value either because it diverges (enters an infinite loop) or because it raises an uncaught exception due to a runtime condition (e.g., division by zero). Type soundness guarantees that if an expression of type  $\tau$  evaluates to a value  $v$ , then  $v$  is guaranteed to be a legitimate member of type  $\tau$ .
- **3. Effect (Side Effects & Behavioral Actions):** Evaluation may engender effects—actions resulting from computation other than returning a value. Effects include raising exceptions, allocating or mutating reference cells, performing file/socket I/O, or communicating across concurrent channels. The static type of an expression classifies only its value, not its side effects.

### 1.3 Core Primitive Types & Overloading Resolution

- **The Integer Type (int):** Represents fixed-precision signed integers (0, 1, 2, ~1, ~2). Negative numbers use the prefix tilde (~) rather than a minus sign. Primitive operations include addition (+), subtraction (-), multiplication (\*), integer division (div), and modulus (mod).
- **The Real Type (real):** Represents floating-point numbers (e.g., 3.14, 2.17, 0.1E6). Operations include +, -, \*, /, and comparisons. Floating-point division uses the forward slash (/) to distinguish it from integer quotient (div).
- **No Implicit Coercion:** ML enforces strict type separation and performs zero implicit type casting. An expression such as `3 + 3.14` is statically rejected. The programmer must explicitly convert types using functions such as `real(3) + 3.14` or `3 + round(3.14)`.
- **Overloading Resolution Mechanics:** Arithmetic operators (+, -, \*) are overloaded for int and real. The compiler resolves overloading based on the context of the nearest enclosing expression. If argument types cannot be unambiguously determined from context, the compiler defaults the operation to type int.
- **The Boolean Type (bool) & Conditionals:** Values consist of true and false. The conditional expression 'if exp1 then exp2 else exp3' requires exp1 to have type bool, and forces exp2 and exp3 to possess identical types. Short-circuit conjunction (andalso) and disjunction (orelse) are syntactic sugar for conditional expressions.
- **Character (char) & String (string) Types:** Characters are written as `#"a"`, `#"b"`. Strings are immutable sequences of characters (e.g., "hello"). Built-in operations include concatenation (^), length calculation (size), and conversions (`String.explode : string -> char list`, `String.implode : char list -> string`).

## 2. Declarations, Bindings, Lexical Scoping & Product Types

The foundational building block for program construction in ML is the declaration. Declarations introduce identifiers into the static environment, binding names to types, values, or constructors.

### 2.1 Variables, Value Bindings & The Substitution Principle

- **Variables Do Not Vary:** In ML, a variable is an immutable identifier bound to a computed value. Once bound via `'val var : typ = exp'`, the binding is permanent throughout its scope. Value binding is fundamentally distinct from imperative variable assignment.
- **Static / Lexical Scoping:** Identifier resolution is determined entirely by static program text rather than dynamic execution history. A variable reference always binds to the nearest lexically enclosing declaration in the program syntax.
- **Shadowing vs. Mutation:** Re-declaring a variable with the same name introduces a new binding that shadows (eclipses) the older binding within the new scope. The original binding is never mutated; if captured inside a previously defined closure, the old binding remains active and unmodified.
- **The Substitution Principle:** Static semantics ensures that any occurrence of a variable in an expression can be conceptually replaced by its bound value prior to evaluation without altering program behavior.

### 2.2 Limiting Scope: Let Expressions & Local Declarations

- **Let Expressions:** The construct `'let dec in exp end'` limits the scope of declarations in dec strictly to the evaluation of expression exp. All bindings introduced within dec are discarded upon completion of exp.
- **Local Declarations:** The construct `'local dec1 in dec2 end'` limits the scope of helper bindings in dec1 strictly to the visibility of declarations in dec2, exporting only dec2 to the enclosing environment.
- **Type and Value Environments:** The compiler maintains a dual compile-time environment: the Type Environment records identifier typing rules and type abbreviations (`'type float = real'`), while the Value Environment records runtime value associations.

## 2.3 Product Types, Tuples & Records

- **Tuples (n-tuples):** Ordered Cartesian product types written as `tau_1 * ... * tau_n`. Values are ordered sequences `(val_1, ..., val_n)`. Parenthesization is structurally significant: a pair of pairs `((int * int) * (real * real))` is distinct from a 4-tuple `(int * int * real * real)`.
- **The Unit Type (0-tuple):** The nullary tuple `()` is the sole value of type `unit`. It represents the absence of meaningful data and is standard for functions executed purely for side effects.
- **Record Types:** Unordered product types where components are accessed via distinct field labels: `{lab_1:tau_1, ..., lab_n:tau_n}`. Record values are written as `{lab_1=val_1, ..., lab_n=val_n}`.
- **Record Patterns & Ellipsis:** Pattern matching on records allows selective field extraction. The ellipsis pattern `{protocol=p, ...}` extracts field `protocol` while ignoring unmentioned fields, provided the surrounding context uniquely determines the full record type.

## 2.4 Clausal Pattern Matching & Case Analysis

- **Homogeneous vs. Heterogeneous Types:** Tuple and record types are homogeneous (every value has identical structure). Base types (`int`, `char`) and datatypes are heterogeneous (values take multiple distinct forms).
- **Clausal Function Syntax:** Functions over heterogeneous types are defined clausally: `'fn pat_1 => exp_1 | ... | pat_n => exp_n'`. Clauses are evaluated in top-to-bottom order; the first pattern matching the argument triggers evaluation of its corresponding body expression.
- **Exhaustiveness Checking:** The ML compiler analyzes pattern coverage to ensure that every possible value of the domain type matches at least one clause. Inexhaustive matches trigger compiler warnings, flagging unhandled edge cases.
- **Redundancy Checking:** The compiler detects and rejects unreachable clauses whose patterns are entirely subsumed by preceding clauses, catching logic errors during compilation.

## 3. First-Class Functions, Closures & Higher-Order Control Patterns

In Standard ML, functions are first-class citizens: they possess the same rights and privileges as primitive data types. Functions can be passed as arguments, returned as computed results, bound to identifiers, and stored inside data structures.

### 3.1 Functions as Values & Call-by-Value Semantics

- **Lambda Expressions (fn):** Anonymous function values are written as `'fn var : typ => exp'`. The syntactic sugar `'fun f (x:typ) : typ = exp'` binds a named function.
- **Call-by-Value Evaluation:** Function application is call-by-value (eager): argument expressions are fully evaluated to values before being bound to the function's formal parameter. Application is represented by juxtaposition `('f x')`.
- **First-Class Status:** Functions can be constructed dynamically at runtime, stored in tuples/records, and yielded as outputs of other functions, forming the basis of functional composition.

### 3.2 Closures and Environment Capture

- **Lexical Closure Architecture:** When a function expression is evaluated, the runtime creates a closure—a data structure pairing the function's executable code with a pointer to the active lexical environment at its definition site.
- **Free Variable Resolution:** All free variables within a function body are permanently bound to their values in the captured environment, regardless of when or where the function is eventually applied.

- **Encapsulated Private State:** Closures provide secure data encapsulation. Inner functions retain exclusive access to variables in their defining scope that have gone out of scope for the rest of the program, mirroring object-oriented private fields.

### 3.3 Currying, Combinators & Staged Computation

- **Currying Transformation:** Currying converts a function taking an n-tuple argument into a chain of single-argument functions: `'curry : ('a * 'b -> 'c) -> 'a -> 'b -> 'c'`, defined as `'fun curry f x y = f (x, y)'`.
- **Staging Computation:** Staging divides computation into 'early' arguments and 'late' arguments. By performing expensive precomputations (such as compiling a search pattern or building an index) as soon as early arguments arrive, the returned closure executes rapidly when applied to subsequent late arguments.
- **Control Pattern Abstraction (Fold / Reduce):** Common list traversal algorithms are unified under the higher-order 'reduce' (or fold) combinator: `'fun reduce (unit, opn, nil) = unit | reduce (unit, opn, h::t) = opn (h, reduce (unit, opn, t))'`. Summation, product, and list concatenation are instances of this single control pattern.

## 4. Polymorphism, Type Inference & The Value Restriction

The ML type system combines static safety with maximum programming convenience via automatic Hindley-Milner type inference and parametric polymorphism.

### 4.1 Principal Typing Property & Unification

- **The Principal Typing Theorem:** For every well-typed expression in ML, there exists a unique, most general type scheme (the principal type) from which all other valid types for that expression can be derived via substitution.
- **Constraint Satisfaction via Unification:** Type inference operates by generating equational constraints among the types of subexpressions and solving them using Robinson's syntactic unification algorithm (analogous to Gaussian elimination).
- **Type Schemes & Type Variables:** Polymorphic types are expressed using type variables ('a', 'b'). For example, the polymorphic identity function `'fn x => x'` is assigned the principal type scheme `"a -> 'a"`, indicating it operates uniformly across all types.

### 4.2 The Value Restriction on Polymorphic Declarations

- **The Soundness Dilemma:** In the presence of mutable references, naive polymorphic declarations cause catastrophic type safety violations. If a reference cell `'ref nil'` were given the polymorphic type `"a list ref"`, it could be updated with a list of integers and subsequently read as a list of functions, crashing the runtime.
- **The Value Restriction Rule:** A variable introduced by a `val` declaration `'val x = exp'` is granted a polymorphic type scheme if and only if the right-hand side expression `exp` is a syntactic value (e.g., a constant, variable, tuple of values, or `fn` expression).
- **Non-Value Expressions:** Expressions requiring runtime evaluation (such as function applications `'val l = rev nil'` or ref allocations `'val r = ref nil'`) are non-values; their type variables are not generalized and remain monomorphic.
- **Eta-Expansion Workaround:** If a non-value expression evaluates to a function, polymorphism can be restored via eta-expansion: transforming `'val f = g h'` into `'fun f x = g h x'` (since `fn` expressions are syntactic values).

## 5. Inductive Datatypes, Abstract Syntax & Structural Recursion

The datatype declaration is ML's extensible mechanism for introducing new, generative, user-defined algebraic data types.

## 5.1 Concrete Algebraic Datatypes

- **Generative Type Declarations:** A datatype declaration introduces new type constructors and value constructors: 'datatype 'a option = NONE | SOME of 'a'. Datatypes are generative: re-declaring an identically named datatype creates an entirely distinct type incompatible with the old one.
- **Nullary & Value Constructors:** Constructors serve a dual purpose: they act as functions to construct values ('SOME 5') and as patterns to deconstruct values during case analysis ('case opt of NONE => ... | SOME x => ...').
- **Eliminating Null Pointer Exceptions:** By using the parametric 'a option type to represent optional data or partial results, the ML type system forces callers to explicitly handle missing values at compile time, eliminating runtime null-dereference faults.

## 5.2 Recursive Trees, Forests & Mutually Recursive Types

- **Binary Trees:** Defined inductively as 'datatype 'a tree = Empty | Node of 'a tree \* 'a \* 'a tree'. Functions on trees (size, height, traversal) are defined by structural recursion mirroring the datatype clauses.
- **Mutually Recursive Datatypes (Trees and Forests):** Variadic trees (nodes with arbitrary numbers of children) are modeled via mutually recursive definitions: 'datatype 'a tree = Empty | Node of 'a \* 'a forest and 'a forest = None | Tree of 'a tree \* 'a forest'.
- **Abstract Syntax Trees (ASTs):** Datatype declarations provide the standard medium for modeling domain-specific languages and compiler intermediate representations (e.g., arithmetic expressions 'datatype expr = Numeral of int | Plus of expr \* expr | Times of expr \* expr').

## 6. Computational Effects: Dynamic Exceptions, References & Lazy Streams

Standard ML integrates controlled computational effects into its functional foundation, providing language constructs for non-local control transfers, mutable state, and demand-driven evaluation.

### 6.1 Dynamic Exceptions as Extensible Types

- **The Extensible Exception Type (exn):** Exceptions in ML are values of the open, dynamically extensible datatype exn. New exception constructors carrying arbitrary payload values are declared via 'exception SyntaxError of string'.
- **Raising and Handling Exceptions:** The expression 'raise exp' aborts normal evaluation and propagates an exception packet up the call stack until intercepted by an enclosing 'exp handle match' handler.
- **Backtracking Search:** Exceptions provide an efficient mechanism for implementing state-space backtracking (e.g., in the N-Queens problem or coin change solvers). When a search path hits a dead end, raising an exception immediately rolls back execution to the nearest choice point.

### 6.2 Mutable Reference Cells, Identity & Aliasing

- **Reference Primitives:** Mutable cells are created via 'ref : 'a -> 'a ref', dereferenced via '! : 'a ref -> 'a', and updated via infix assignment ':= : 'a ref \* 'a -> unit'.
- **Identity and Pointer Equality:** Reference cells introduce reference equality. Two reference cells r and s are equal (r = s) if and only if they were allocated by the exact same invocation of ref, regardless of their contents.

- **Aliasing Pitfalls:** When multiple variables reference the same mutable cell, updating one variable implicitly mutates the other. Uncontrolled aliasing complicates formal verification and program comprehension.
- **Ephemeral vs. Persistent Data Structures:** Pure functional data structures are persistent (operations produce new versions while preserving old ones). Mutable data structures are ephemeral (operations update in-place, destroying prior states).

### 6.3 Lazy Evaluation, Suspensions & Infinite Streams

- **Eager vs. Lazy Evaluation:** Eager evaluation evaluates expressions immediately upon binding. Lazy evaluation defers computation until the value is explicitly required by an operation, employing memoization (by-need) to evaluate at most once.
- **Suspensions (Thunks):** A suspended computation is modeled as a nullary function `'unit -> 'a'`. Suspensions are evaluated on-demand via `'force : 'a susp -> 'a'`.
- **Infinite Streams:** Streams model unbounded sequences (e.g., the stream of prime numbers via the Sieve of Eratosthenes). A stream is defined as `'datatype lazy 'a stream = Cons of 'a * 'a stream'`, allowing consumers to extract prefixes without evaluating the infinite tail.

## 7. The ML Module System: Signatures, Structures & Ascription

The Standard ML module language provides advanced architectural mechanisms for structuring large software systems, enforcing data abstraction, and supporting parameterized code reuse.

### 7.1 Signatures, Structures & Path Resolution

- **Signatures as Module Types:** A signature (`'signature SIG = sig ... end'`) specifies the public interface of a module, declaring required types, datatypes, exceptions, and value type schemes.
- **Structures as Module Implementations:** A structure (`'structure Strid = struct ... end'`) encapsulates concrete definitions of types and values. Components are accessed via qualified paths (`'Strid.val_name'`).
- **Many-to-Many Architecture:** In ML, a single signature may be implemented by dozens of independent structures, and a single structure can simultaneously satisfy multiple distinct signatures.

### 7.2 Signature Matching, Ascription & Data Abstraction

- **Principal Signatures & Subtyping:** A structure matches a target signature if its principal signature provides all required components with at least as general types. Structures may provide extra components not required by the signature.
- **Opaque Ascription (`>`):** Opaque ascription (`'structure S :> SIG = struct ... end'`) enforces strict data abstraction. Types declared without definitions in SIG are made totally abstract: outside clients cannot inspect or rely upon their underlying representation.
- **Transparent Ascription (`:`):** Transparent ascription (`'structure S : SIG = struct ... end'`) prunes unmentioned components but preserves full transparency of type definitions, exposing their concrete identities to clients.
- **The Rely-Guarantee (Assume-Ensure) Method:** Opaque ascription confines representation invariants to the module implementation. Operations assume the invariant holds for input arguments and guarantee it holds for outputs; clients cannot forge invalid states.

### 7.3 Substructures, Sharing Constraints & Functors

- **Substructures & Module Hierarchies:** Structures can contain nested substructures, enabling hierarchical system decomposition (e.g., a dictionary module containing an ordered Key substructure).



- **The Module Coherence Problem:** When combining independent modules (e.g., Points, Vectors, Spheres in a Geometry package), opaque ascription causes identical underlying types to become incompatible abstract types.
- **Sharing Constraints (sharing type):** The specification 'sharing type S1.t = S2.t' forces the compiler to verify that two abstract types in different substructures are identical, restoring type coherence across module boundaries.
- **Functors (Parameterized Modules):** A functor is a module-level function taking one or more structures as arguments and returning a new structure: 'functor DictFun (structure K : ORDERED) :> DICT = struct ... end'. Functors enable generic data structures parameterized over types and their behavioral operations.

## 8. Verification, Proof-Directed Debugging & Persistent Data Structures

Standard ML integrates formal program verification directly with software engineering practice, leveraging structural induction, amortized complexity analysis, and self-balancing tree invariants.

### 8.1 Mathematical & Structural Induction Principles

- **Mathematical Induction over Integers:** Proving a property  $P(n)$  for non-negative integers requires verifying the base case  $P(0)$  and the inductive step  $P(k) \Rightarrow P(k+1)$ . This justifies total correctness of recursive functions like factorial and fast exponentiation.
- **Structural Induction over Lists and Trees:** To prove property  $P(l)$  holds for all lists  $l$ , it suffices to prove  $P(\text{nil})$  holds, and that  $P(t) \Rightarrow P(h::t)$ . For trees, proving  $P(\text{Empty})$  and  $P(\text{lft}) / P(\text{rht}) \Rightarrow P(\text{Node}(\text{lft}, v, \text{rht}))$  establishes correctness for all binary trees.
- **Total vs. Partial Correctness:** Partial correctness proves that if a function terminates, its output satisfies the post-condition. Total correctness proves both termination and post-condition compliance for all valid inputs.

### 8.2 Proof-Directed Debugging: The Regular Expression Matcher

- **The Continuation-Passing Matcher:** A regular expression matcher 'match : regexp -> char list -> (char list -> bool) -> bool' decomposes pattern matching using failure continuations to handle concatenation and alternation.
- **Uncovering Subtle Non-Termination:** Attempting an inductive proof of termination for iterated expressions ( $r^*$ ) reveals a flaw: if  $r$  accepts the null string epsilon, the matcher loops infinitely without advancing through the input string.
- **Standard Regular Expressions:** Proof-directed debugging resolves this bug by transforming regular expressions into standard form  $r = \text{delta}(r) + r^\wedge$ , where  $r^\wedge$  cannot match epsilon, restoring guaranteed termination.

### 8.3 Amortized Analysis of Persistent Queues

- **Two-List Queue Representation:** A persistent FIFO queue is represented as a pair of lists (back, front), where elements arrive at back and depart from front. When front is exhausted, back is reversed and installed as front.
- **The Invariant:** Representation invariant: front is empty only if back is empty. This is enforced by a smart constructor 'make\_queue'.
- **Amortized Charge Scheme:** Using the Banker/Physicist method, each element is charged 2 credits upon insertion (1 for cons'ing onto back, 1 deposited as savings). When reversal occurs, accumulated credits pay for the  $O(k)$  reversal steps. Thus, every queue operation executes in  $O(1)$  amortized worst-case time.
-

## 8.4 Self-Balancing Red-Black Trees

- **Red-Black Tree Invariants:** A binary search tree where each node is colored Red or Black satisfying: (1) Children of a Red node are Black, and (2) Every path from a node to a leaf contains the same number of Black nodes (Black Height  $h$ ).
- **Logarithmic Height Guarantee:** A red-black tree with  $n$  nodes has height bounded by  $2 * \lg(n + 1)$ . Consequently, lookup, insertion, and deletion execute in guaranteed  $O(\lg n)$  worst-case time.
- **Pattern-Matched Rebalancing Rotations:** Insertion preserves black height but may introduce a Red-Red violation. Four rotation cases are concisely matched and restored to balanced form in constant time using ML pattern matching.